SECTION 1, PART 1 CHEAT SHEET

Ready-made models never saw our tickets

`pipeline` runs a model from the Hub in one call. A trained classifier answers only with its own labels, zero-shot puts our four teams into sentences, and a chat model answers in words. The best of them routed 57.5 % of the 40 tickets: the reason to fine-tune.

tickets.csv, the first rows

text	gold answer
The mobile app crashes …	technical
Our whole team is locked out …	technical
Can you tell me the price …	sales
Please change the email …	account

texts the models read **gold** kept apart

1 THE DATA

40 tickets, one right team each

Each ticket has its text and the team that should handle it, the gold answer. The models read only the text.

```
tickets = pd.read_csv("data/tickets.csv")
tickets.head()
tickets["department"].value_counts()
texts = tickets["text"].tolist()
gold = tickets["department"].tolist()
```

COURSE RUN **40** tickets: technical **12**, billing **11**, account **9**, sales **8**.

2 ONE CALL

pipeline: a task, a model, a device

The pipeline hides the tokenizer, the model and the tidy-up. A trained classifier answers only with the labels it learned from its training examples: SST-2 movie reviews give two, and our four teams have nowhere to land.

COURSE RUN `NEGATIVE` **0.991** on the duplicate charge. On all 40 tickets, **37** came back `NEGATIVE`.

Please refund the duplicate payment.
ticket 0, the duplicate charge

distilbert / **distilbert-base-uncased-finetuned-sst-2-english**

NEGATIVE 0.991 | POSITIVE

our four teams have no slot to land in:
billing, technical, account, sales

```
from transformers import pipeline, logging
sentiment = pipeline("text-classification",
                     model="distilbert/distilbert-base-uncased-finetuned-sst-2-english",
                     device_map="cuda")
response = sentiment(texts[0])
results = sentiment(texts)
```

3 ITS OWN LABELS

28 emotions, none of them a team

A model trained on Reddit comments (GoEmotions) knows 28 feelings. `top_k=3` passes through `pipeline` and returns the three best, best first.

```
emotions = pipeline("text-classification",
    model="SamLowe/roberta-base-go_emotions",
    top_k=3, device_map="cuda")
emotions(texts[10])
top.append(r[0]["label"])
```

COURSE RUN the sarcastic ticket read as admiration **0.825**. Over all 40 the top emotion was neutral **25** times.

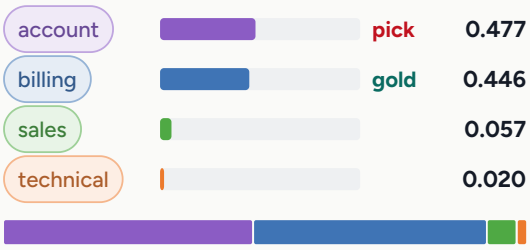
Great, another update and now nothing syncs.

SamLowe / **roberta-base-go_emotions**

28 labels, the top three:

admiration	0.825
love	0.585
joy	0.088

each label goes into one sentence:
`"This support ticket is about {}."`



one shared bar: the four add up to 1

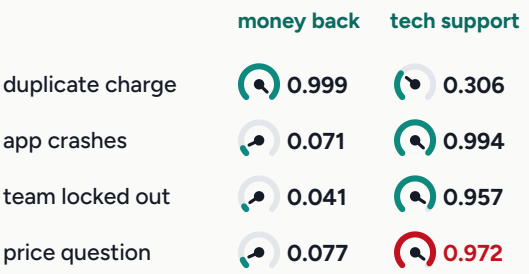
4 ZERO-SHOT

Each label becomes a sentence

An NLI model scores the ticket against one sentence per label. The labels come back sorted, so `labels[0]` is the pick. Any labels, no training.

```
zero_shot = pipeline("zero-shot-classification",
    model="facebook/bart-large-mnli", device_map="cuda")
DEPTS = ["billing", "technical", "account", "sales"]
response = zero_shot(texts[0], candidate_labels=DEPTS,
    hypothesis_template="This support ticket is about {}.")
```

COURSE RUN ticket 0 went to account, gold billing. On all 40: **0.575** right (**23 of 40**), average top score **0.588**.



two separate yes or no scores per ticket

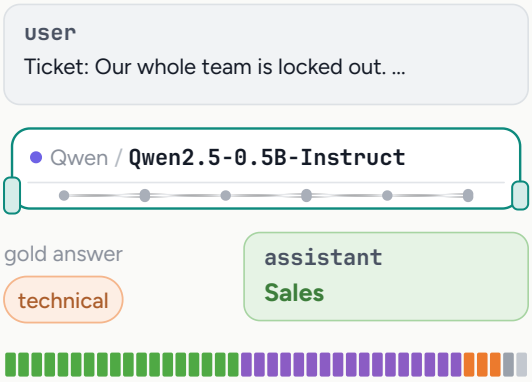
5 MULTI_LABEL

One yes or no per label

`multi_label=True` scores each label on its own, so the scores no longer add up to 1. A score above 0.5 counts as a yes.

```
results = zero_shot(texts,
    candidate_labels=["asks for money back"],
    hypothesis_template="This support ticket {}.",
    multi_label=True)
tickets["refund_pred"] = tickets["refund_score"] > 0.5
```

COURSE RUN the refund question on all 40: **0.95** right, all **6** refunds found, **2** false yeses.



40 replies: sales 18, account 17, technical 3, team 1, accounting 1, billing never

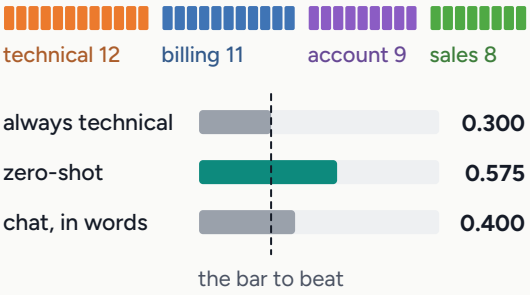
6 A CHAT MODEL

Words in, words out

A message is a dict with a `role` and its `content`. The reply is the last message: a word with no score, which can fall outside the list, so tidy it before comparing.

```
chat = pipeline("text-generation",
    model="Qwen/Qwen2.5-0.5B-Instruct", device_map="cuda")
msgs = [{"role": "user", "content": question}]
out = chat(msgs, max_new_tokens=20, do_sample=False)
reply = out[0]["generated_text"][-1]["content"]
answers.append(reply.strip().strip(".").lower())
```

COURSE RUN right on **16 of 40** tickets (**0.4**), and never `billing`.



7 SCORING

Every model against the gold answers

Accuracy is the share of picks that match the gold answer. Naming the biggest team every time is the bar to beat, and none of the three models was trained on our tickets.

```
scoreboard = {
    "always 'technical'": ...,
    "zero-shot, our four labels": ...,
    "chat model, in words": ...
}
pd.Series(scoreboard)
```

zero-shot pick

gold	account	billing	sales	technical
account	7	1	0	1
billing	4	4	2	1
sales	4	3	1	0
technical	1	0	0	11

the diagonal is right; account took 4 billing and 4 sales tickets

8 READ THE MISTAKES

Where the misses went

`pd.crosstab` counts gold team against pick. The diagonal is right; every other cell names a mix-up, and the mix-ups show what a model cannot tell apart.

```
pd.crosstab(tickets["department"], tickets["zero_shot"])
pd.crosstab(tickets["department"], tickets["chat"])
```

COURSE RUN zero-shot got **11 of 12** technical tickets but **4 of 11** billing; the chat model sent **6 of 11** billing tickets to sales.

TERMS

pipeline: one call: the tokenizer, the model and the tidy-up.

the Hub: the Hugging Face Hub, where models are shared.

gold answer: the right team for a ticket, kept apart to score the picks.

zero-shot classification: each label put in a sentence and judged by an NLI model.

`multi_label`: one yes or no score per label; the scores do not share.

WHAT EACH CALL HANDS BACK

`sentiment(texts[0])`
a list with one dict: **label**, **score**

`emotions(texts[10])`
with `top_k=3`: the top three, best first

`zero_shot(text, ...)`
sequence, **labels**, **scores**, strongest first

`chat(msgs, ...)`
generated_text: the messages, the reply last

`pd.crosstab(gold, picks)`
a table of counts, gold team by pick

WATCH OUT

- A classifier answers only with its own labels: the price question came back `NEGATIVE` **0.993**.
- `multi_label` gave the price question **0.972** on "asks for technical support".
- The chat model's replies fell outside the list: **team** and **accounting**.